

Week 11 Assessment Report

Graduation Profile Setup — Docker, CI/CD and Production-Style Sandbox

Project: Phase-2 **Student:** Akshay

1. Week 11 Objective

The Week 11 work focused on preparing the Phase-2 backend for a production-style sandbox by containerizing the application, using a multi-stage Docker build, configuring continuous integration with GitHub Actions, establishing service connectivity, and protecting local secrets and runtime-only files from source control.

2. Implementation Completed

Area	Implementation / Verification
Multi-stage Dockerization	Created a two-stage node:24-alpine Dockerfile. Production dependencies are installed with <code>npm ci --omit=dev</code> and copied into the production stage.
Docker image	Built <code>phase2-week11:latest</code> successfully. Reported Docker content size was approximately 68.2 MB.
Container deployment	Started <code>phase2-week11-container</code> with port 3000 exposed and HTTPS certificates mounted at runtime.
HTTPS	Application verified at <code>https://localhost:3000</code> . The private key is not copied into the image.
Redis service	Created <code>phase2-network</code> and connected <code>phase2-week11-container</code> with <code>week7-redis</code> . Redis connection completed successfully.
CI/CD	Created <code>.github/workflows/ci.yml</code> to checkout code, set up Node.js 24, run <code>npm ci</code> , run tests, and build the Docker image.
Security	Configured <code>.gitignore</code> to exclude <code>.env</code> , <code>node_modules</code> , logs, local certificates, and the SQLite database.

3. Dockerfile Structure

Dependencies stage: node:24-alpine, /app working directory, package manifests copied, production dependencies installed with `npm ci --omit=dev`.

Production stage: node:24-alpine, runtime dependencies and application files copied, port 3000 exposed, service started with `node app.js`.

4. Container and Service Verification

Check	Result
Docker	Version 29.4.3
Image build	Successful
Node container	<code>phase2-week11-container</code> — Up
Port	<code>0.0.0.0:3000 -> 3000/tcp</code>
Redis	<code>week7-redis</code> — Up
Network	<code>phase2-network</code>
Redis connection	Redis socket connected; Redis client ready; Redis connection ready
Application	Server running on <code>https://localhost:3000</code>
Metrics	<code>https://localhost:3000/metrics</code> verified

5. Testing and CI Configuration

The local Jest run completed successfully: 1 test suite passed and 1 test passed. Jest displayed a module naming collision warning involving tests/package.json, but the test result was successful. The CI workflow reproduces dependency installation, testing, and Docker image building for pushes and pull requests to main.

CI sequence: checkout → Node.js 24 → npm ci → npm test → docker build -t phase2-week11 .

6. Repository and Security Verification

Before committing, the staged file list was checked for sensitive names. No staged file matched env, key, cert, db, or node_modules. The local SSL certificates, environment file, logs, dependencies, and SQLite database remain outside the committed project.

7. Git Verification

Commit	Message
a3c0339	Complete Week 11 Docker CI/CD setup
3e2347d	Ignore local certificates and database

Final local Git status: nothing to commit, working tree clean. The GitHub remote was configured as <https://github.com/Aksha4/Phase-2.git>. A remote push was not completed during this session.

8. Deliverables Checklist

- ✓ Multi-stage Dockerfile created and built
- ✓ Production container started successfully
- ✓ Redis service connected through Docker network
- ✓ HTTPS certificate mounted at runtime
- ✓ GitHub Actions CI workflow created
- ✓ npm test completed successfully
- ✓ Docker build included in CI workflow
- ✓ Sensitive local files excluded from Git
- ✓ Week 11 work committed locally
- ✓ Working tree verified clean

9. Conclusion

Week 11 established a reproducible containerized deployment and CI foundation for the Phase-2 backend. The application was built into a multi-stage Docker image, executed as a container with runtime HTTPS certificates, connected to Redis over a dedicated Docker network, and prepared for automated test-and-build verification through GitHub Actions. Source-control checks also kept credentials, certificates, logs, dependencies, and the SQLite database outside the committed project.